

```
$ script -q -e
$ ls invalid_file      # Provide non-existing file name.
ls: cannot access invalid_file: No such file or directory
$ exit
exit
$ echo $?              # Check exit status.
2
```

The '-t' or '--timing' option dumps timing data to the standard error stream. This timing data contains two space-separated fields. The first field indicates how much time has elapsed since the previous output, and the second field indicates how many characters were printed this time. Sounds confusing? For now, just remember that it stores some timing information. Later on, in the script recipes section we will look at how this information is useful to play back terminal sessions.

When we invoke the script command with the '-f' or '--flush' options, it flushes the output after each write operation. In the script recipes section, we will see how to leverage this behaviour to broadcast terminal sessions.

Script recipes

We are now familiar with the script command and we have also looked at a few examples. Let us now discuss a few recipes related to this command.

Recipe 1. Record and play back terminal sessions: We can record the order and timing of the command into a plain text file. Later on, we can revisit it by replaying it. Timing information ensures that the output happens at the same speed as it originally appeared when the script was recorded. Trust me, you will be amazed at what you see.

To start the terminal recording, execute the following command:

```
$ script -q -a output.log -t 2> timing.log # Redirect
"stderr" to store timing data in file.
$ hostname
minty
$ uname
Linux
$ lsb_release -i
Distributor ID:   LinuxMint
$ exit
```

exit

Now replay the recording by using the 'scriptreplay' command. Both the '--typescript' and '--timing' options are self-explanatory.

```
$ scriptreplay --typescript output.log --timing timing.log
```

Recipe 2. Speed up the replay display: The 'scriptreplay' command provides the '-d' or '--divisor' option, which is used to speed up the replay display by the number of times specified.

```
$ scriptreplay --typescript output.log --timing timing.log
--divisor 4
```

The above command will speed up the replay display by four times.

Recipe 3. Broadcast the terminal session using script and named pipes: We can achieve good tele-cooperation by taking the following simple steps.

1. Open two separate terminals (it would be better if you use terminals instead of tabs).
2. In the first terminal, create a named pipe and invoke the *Script* command with the '-f' or '--flush' options:

```
$ mkfifo broadcast.fifo
$ script -f broadcast.fifo
```

3. Switch to the second terminal and perform the 'cat' operation on the named pipe:

```
$ cat broadcast.fifo
```

4. Switch back to the first terminal and type a few commands. You will observe that anything typed on the first terminal will be broadcast to the second terminal. To stop broadcasting, quit the script session. 

By Narendra Kangralkar

The author is a FOSS enthusiast and loves exploring anything related to open source. He can be reached at narendrakangralkar@gmail.com.

**India's #1 Website
For Electronics
Engineers Working
Across The Globe**

Established: 1998
Registered Users: 300,000+



electronicsforu.com
If it's electronics, it's here

Monthly Statistics Page Impressions: 750,000+
Unique Techies: 150,000

For any queries, please contact our team at
efyeng@efyindia.com OR +91-11-26610601

